DATASCI 266 Final Project

Comparing Traditional, Multilingual, and Korean-Specific Models for Korean Sentiment Classification using KOTE

0. Environment Setup

```
!pip -q install transformers datasets accelerate sentencepiece sacremos

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from collections import Counter

from datasets import load_dataset, Dataset

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, f1_score, classification_re

from transformers import AutoTokenizer, AutoModelForSequenceClassificat
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 897.5/897.5 kB 22.6 MB/s e
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

1. Load Dataset

This section loads the KOTE dataset from Hugging Face and converts the train, validation, and test splits to pandas DataFrames for inspection and preprocessing.

```
dataset = load_dataset(
    "searle-j/kote",
    revision="refs/convert/parquet"
)

train_ds = dataset["train"]
val_ds = dataset["validation"]
test_ds = dataset["test"]

df_train = train_ds.to_pandas()
df_val = val_ds.to_pandas()
df_test = test_ds.to_pandas()

print(dataset)
print(df_train.head())
```

```
dichotomized/train/0000.parquet: 100%                    4.75M/4.75M [00:04<00:00, 23.9MB/s]

0000.parquet: 100%                                       607k/607k [00:00<00:00, 14.9MB/s]

0000.parquet: 100%                                       599k/599k [00:00<00:00, 10.1MB/s]

Generating train split:        40000/0 [00:00<00:00, 159529.19 examples/s]

Generating validation split:        5000/0 [00:00<00:00, 44971.32 examples/s]

Generating test split:        5000/0 [00:00<00:00, 40424.45 examples/s]
DatasetDict({
    train: Dataset({
        features: ['ID', 'text', 'labels'],
        num_rows: 40000
    })
    validation: Dataset({
        features: ['ID', 'text', 'labels'],
        num_rows: 5000
    })
    test: Dataset({
        features: ['ID', 'text', 'labels'],
        num_rows: 5000
```

```
    })
})
      ID                                                text  \
0  39087                 내가 톰행크스를 좋아하긴 했나보다... 초기 영화 빼고는 다 봤네.
1  30893  정말 상상을 초월하는 무개념 진상들 상대하다 우울증, 공항장애 걸리는 공무원 많아
2  45278  새로운 세상과 조우한 자의 어린아이 같은 반응, 어쩌면 회복된 것은 눈이 아닌 순수
3  16398        미역은 원생생물계 산호초는 동물ㅇㅇ 아 미역이 바다의 새ㄱㅇㄱㅋㅋㅋㅋㅋㅋ=
4  13653                           네 맞습니다 플스는 역시 30프레임이 어울리죠 ㅎ

                                            labels
0                            [2, 13, 15, 16, 29, 39]
1              [0, 5, 7, 10, 19, 22, 29, 35, 36, 38]
2                                        [1, 2, 7]
3                        [9, 15, 20, 23, 26, 28, 29]
4  [1, 2, 8, 9, 11, 13, 15, 16, 28, 29, 32, 40, 42]
```

```
df_train.to_csv("df_train_processed.csv", index=False)
df_val.to_csv("df_val_processed.csv", index=False)
df_test.to_csv("df_test_processed.csv", index=False)
```

```
df_train.to_csv("/content/drive/MyDrive/df_train_processed.csv", inde
df_val.to_csv("/content/drive/MyDrive/df_val_processed.csv", index=Fa
df_test.to_csv("/content/drive/MyDrive/df_test_processed.csv", index=
```

2. Inspect Label Metadata

The original KOTE labels are represented as integer IDs. This section recovers the label names from the Hugging Face metadata.

```
label_names = train_ds.features["labels"].feature.names
id_to_label = {i: name for i, name in enumerate(label_names)}

print(train_ds.features["labels"])
print(id_to_label)
```

```
List(ClassLabel(names=['불평/불만', '환영/호의', '감동/감탄', '지긋지긋', '고마움
{0: '불평/불만', 1: '환영/호의', 2: '감동/감탄', 3: '지긋지긋', 4: '고마움', 5: '
```

3. Consolidate 43 Emotion Labels into 3 Sentiment Classes

The original KOTE dataset contains 43 fine-grained emotion labels. Following Appendix A of Jeon et al. (2024), the labels are consolidated into three broader categories: negative, positive, and neutral.

```
label_to_sentiment = {
    # Negative
    '불평/불만': 'negative',
    '지긋지긋': 'negative',
    '슬픔': 'negative',
    '화남/분노': 'negative',
    '안타까움/실망': 'negative',
    '의심/불신': 'negative',
    '부끄러움': 'negative',
    '공포/무서움': 'negative',
    '절망': 'negative',
    '한심함': 'negative',
    '역겨움/징그러움': 'negative',
    '짜증': 'negative',
    '어이없음': 'negative',
    '패배/자기혐오': 'negative',
    '귀찮음': 'negative',
    '힘듦/지침': 'negative',
    '죄책감': 'negative',
    '증오/혐오': 'negative',
    '당황/난처': 'negative',
    '경악': 'negative',
    '부담/안_내킴': 'negative',
    '서러움': 'negative',
    '재미없음': 'negative',
    '불쌍함/연민': 'negative',
    '불안/걱정': 'negative',

    # Positive
    '환영/호의': 'positive',
    '감동/감탄': 'positive',
    '고마움': 'positive',
    '존경': 'positive',
```

```
        '기대감': 'positive',
        '뿌듯함': 'positive',
        '편안/쾌적': 'positive',
        '신기함/관심': 'positive',
        '아껴주는': 'positive',
        '즐거움/신남': 'positive',
        '흐뭇함(귀여움/예쁨)': 'positive',
        '행복': 'positive',
        '기쁨': 'positive',
        '안심/신뢰': 'positive',

        # Neutral
        '우쭐댐/무시함': 'neutral',
        '비장함': 'neutral',
        '없음': 'neutral',
        '깨달음': 'neutral',
        '놀람': 'neutral',
    }
```

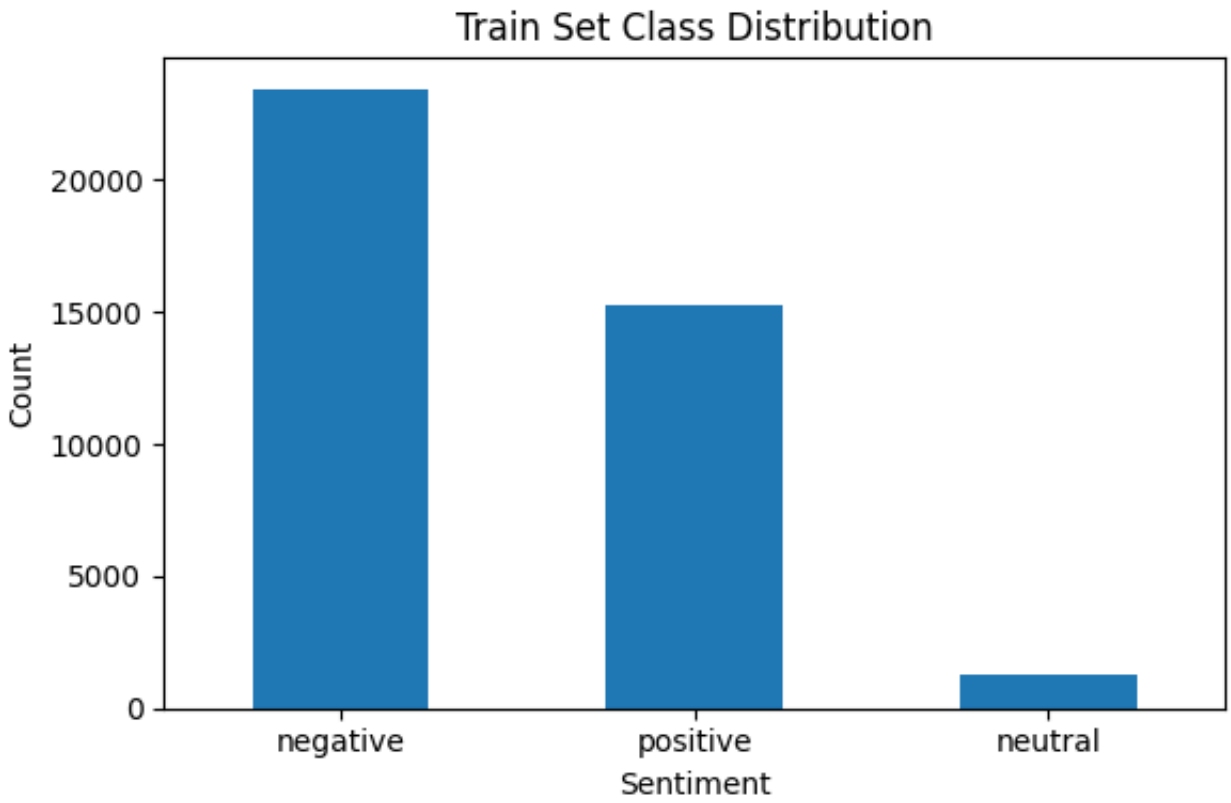
```
score_map = {"negative": -1, "neutral": 0, "positive": 1}

def collapse_labels_score(label_ids):
    sentiments = [label_to_sentiment[id_to_label[i]] for i in label_i
    score = sum(score_map[s] for s in sentiments)
    if score > 0:
        return "positive"
    elif score < 0:
        return "negative"
    else:
        return "neutral"
```

```
df_train["sentiment"] = df_train["labels"].apply(collapse_labels_scor
df_val["sentiment"] = df_val["labels"].apply(collapse_labels_score)
df_test["sentiment"] = df_test["labels"].apply(collapse_labels_score)
```

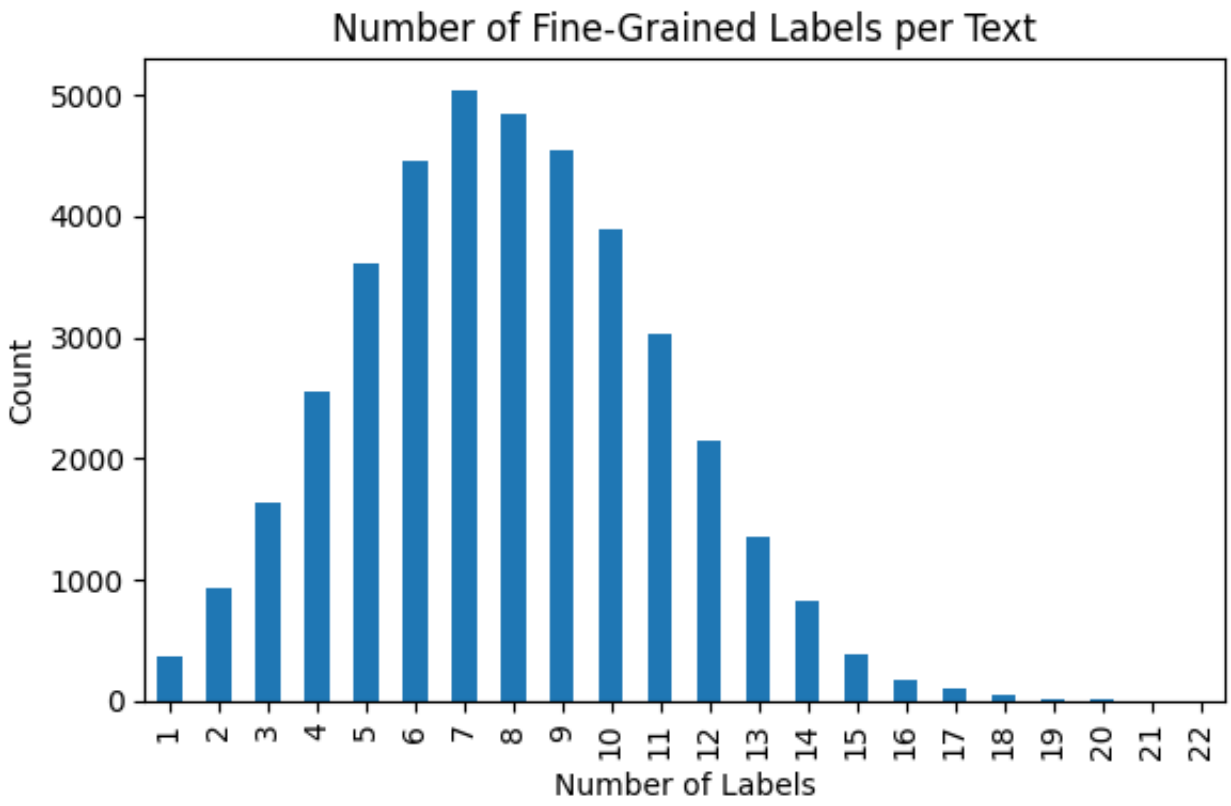
```
sentiment_to_id = {"negative": 0, "positive": 1, "neutral": 2}

df_train["y"] = df_train["sentiment"].map(sentiment_to_id)
df_val["y"] = df_val["sentiment"].map(sentiment_to_id)
df_test["y"] = df_test["sentiment"].map(sentiment_to_id)
```

4. Exploratory Data Analysis (EDA)

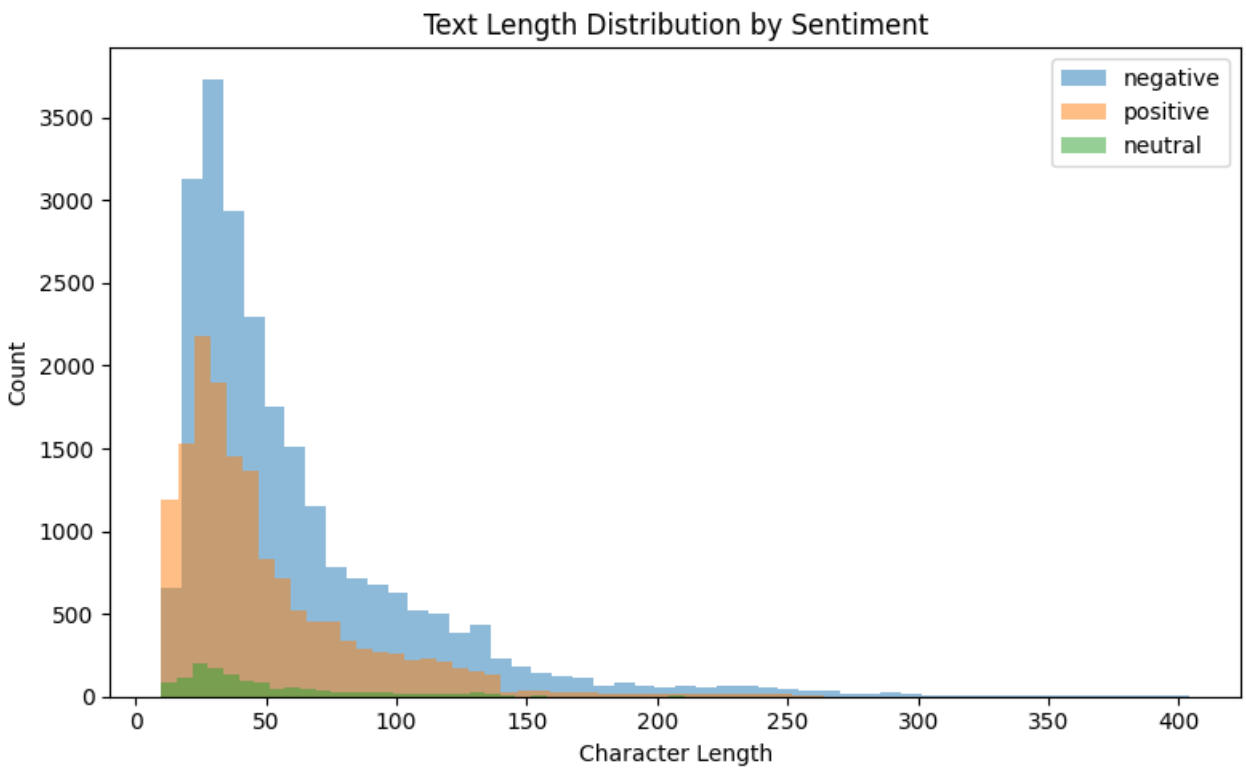
```
df_train["sentiment"].value_counts().plot(kind="bar", figsize=(6,4))
plt.title("Train Set Class Distribution")
plt.xlabel("Sentiment")
plt.ylabel("Count")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



```
df_train["num_labels"] = df_train["labels"].apply(len)

df_train["num_labels"].value_counts().sort_index().plot(kind="bar", f
plt.title("Number of Fine-Grained Labels per Text")
plt.xlabel("Number of Labels")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```



```
df_train["text_len"] = df_train["text"].apply(len)

plt.figure(figsize=(8,5))
for sentiment in ["negative", "positive", "neutral"]:
    subset = df_train[df_train["sentiment"] == sentiment]["text_len"]
    plt.hist(subset, bins=50, alpha=0.5, label=sentiment)

plt.title("Text Length Distribution by Sentiment")
plt.xlabel("Character Length")
plt.ylabel("Count")
plt.legend()
plt.tight_layout()
plt.show()
```



```
print(df_train["num_labels"].describe())
print(df_train.groupby("sentiment")["text_len"].describe()[["mean", "s
```

```
count    40000.000000
mean         7.911475
std          3.098085
min          1.000000
25%          6.000000
50%          8.000000
75%         10.000000
max         22.000000
Name: num_labels, dtype: float64
                mean        std   min    max
sentiment
negative   62.008786  48.726013  10.0  404.0
neutral    51.161585  38.525371  10.0  305.0
positive   50.329287  37.679431  10.0  320.0
```


```
print(df_train["num_labels"].describe())
print(df_train.groupby("sentiment")["text_len"].describe()[["mean", "s
```

```
count    40000.000000
mean         7.911475
std          3.098085
min          1.000000
25%          6.000000
50%          8.000000
75%         10.000000
max         22.000000
Name: num_labels, dtype: float64
                mean        std   min    max
sentiment
negative   62.008786  48.726013  10.0  404.0
neutral    51.161585  38.525371  10.0  305.0
positive   50.329287  37.679431  10.0  320.0
```


5. TF-IDF + Logistic Regression Baseline

A traditional bag-of-words baseline is built using TF-IDF features and multiclass logistic regression.

```
X_train = df_train["text"]
y_train = df_train["y"]

X_val = df_val["text"]
y_val = df_val["y"]

X_test = df_test["text"]
y_test = df_test["y"]
```

```
baseline = Pipeline([
    ("tfidf", TfidfVectorizer(
        max_features=20000,
        ngram_range=(1, 2),
        min_df=2
    )),
    ("clf", LogisticRegression(
        max_iter=2000,
        class_weight="balanced"
    ))
])
```

```
baseline.fit(X_train, y_train)

val_pred = baseline.predict(X_val)
test_pred = baseline.predict(X_test)

print("Validation Accuracy:", accuracy_score(y_val, val_pred))
print("Validation Macro-F1:", f1_score(y_val, val_pred, average="macro

print("\nTest Accuracy:", accuracy_score(y_test, test_pred))
print("Test Macro-F1:", f1_score(y_test, test_pred, average="macro"))
```

```
Validation Accuracy: 0.7042
Validation Macro-F1: 0.5190351295110717

Test Accuracy: 0.6904
Test Macro-F1: 0.4988920302721864
```

```
label_names_3 = ["negative", "positive", "neutral"]

print(classification_report(y_test, test_pred, target_names=label_name
print(confusion_matrix(y_test, test_pred))
```

```
              precision    recall  f1-score   support

    negative       0.79      0.76      0.77      2978
    positive       0.69      0.63      0.66      1865
     neutral       0.04      0.12      0.06       157

    accuracy                           0.69      5000
   macro avg       0.51      0.50      0.50      5000
weighted avg       0.73      0.69      0.71      5000

[[2253  473  252]
 [ 513 1180  172]
 [  74   64   19]]
```

6. Prepare Hugging Face Datasets for Transformer Models

```
train_hf = Dataset.from_pandas(df_train[["text", "y"]].reset_index(dr
val_hf = Dataset.from_pandas(df_val[["text", "y"]].reset_index(drop=T
test_hf = Dataset.from_pandas(df_test[["text", "y"]].reset_index(drop
```

```
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=1)
    return {
        "accuracy": accuracy_score(labels, preds),
        "macro_f1": f1_score(labels, preds, average="macro")
    }
```

7. KLUE-RoBERTa

This section fine-tunes a Korean-specific pretrained transformer model on the consolidated 3-class sentiment task.

```
model_name = "klue/roberta-base"
tokenizer = AutoTokenizer.from_pretrained(model_name)

def tokenize_function(examples):
    return tokenizer(
        examples["text"],
        truncation=True,
        padding="max_length",
        max_length=128
    )

train_tok = train_hf.map(tokenize_function, batched=True)
val_tok = val_hf.map(tokenize_function, batched=True)
test_tok = test_hf.map(tokenize_function, batched=True)

train_tok = train_tok.rename_column("y", "labels")
val_tok = val_tok.rename_column("y", "labels")
test_tok = test_tok.rename_column("y", "labels")

train_tok.set_format(type="torch", columns=["input_ids", "attention_ma
val_tok.set_format(type="torch", columns=["input_ids", "attention_masl
test_tok.set_format(type="torch", columns=["input_ids", "attention_mas
```

```
config.json: 100%                                   546/546 [00:00<00:00, 6.10kB/s]

tokenizer_config.json: 100%                         375/375 [00:00<00:00, 6.30kB/s]

tokenizer.json:        752k/? [00:00<00:00, 7.35MB/s]

special_tokens_map.json: 100%                       173/173 [00:00<00:00, 2.77kB/s]

Map: 100%                                 40000/40000 [00:34<00:00, 1304.11 examples/s]

Map: 100%                                  5000/5000 [00:04<00:00, 1315.39 examples/s]

Map: 100%                                  5000/5000 [00:02<00:00, 1691.76 examples/s]
```

```
model = AutoModelForSequenceClassification.from_pretrained(
```

```
model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=3
)

training_args = TrainingArguments(
    output_dir="./klue_roberta_results",
    eval_strategy="epoch",
    save_strategy="epoch",
    logging_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=3,
    weight_decay=0.01,
    load_best_model_at_end=True,
    metric_for_best_model="macro_f1",
    greater_is_better=True,
    report_to="none"
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_tok,
    eval_dataset=val_tok,
    compute_metrics=compute_metrics
)

trainer.train()
```

model.safetensors: 100% 443M/443M [00:09<00:00, 76.4MB/s]

Loading weights: 100% 197/197 [00:00<00:00, 334.75it/s, Materializing param=roberta.encoder.layer.

```
RobertaForSequenceClassification LOAD REPORT from: klue/roberta-base
Key                              | Status     |
---------------------------------+------------+-
lm_head.dense.weight             | UNEXPECTED |
lm_head.dense.bias               | UNEXPECTED |
lm_head.bias                     | UNEXPECTED |
lm_head.layer_norm.weight        | UNEXPECTED |
lm_head.layer_norm.bias          | UNEXPECTED |
roberta.embeddings.position_ids  | UNEXPECTED |
classifier.dense.bias            | MISSING    |
classifier.dense.weight          | MISSING    |
classifier.out_proj.bias         | MISSING    |
```

```
classifier.out_proj.weight      | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/archit
- MISSING       :those params were newly initialized because missing fr
```

[7500/7500 47:52, Epoch 3/3]

Epoch	Training Loss	Validation Loss	Accuracy	Macro F1
1	0.430524	0.410822	0.852800	0.569721
2	0.295720	0.447586	0.861400	0.592391
3	0.205765	0.547262	0.864200	0.617473

Writing model shards: 100% 1/1 [00:08<00:00, 8.22s/it]

Writing model shards: 100% 1/1 [00:10<00:00, 10.91s/it]

Writing model shards: 100% 1/1 [00:11<00:00, 11.07s/it]

```
There were missing keys in the checkpoint model loaded: ['roberta.embec
There were unexpected keys in the checkpoint model loaded: ['roberta.en
TrainOutput(global_step=7500, training_loss=0.3106693115234375,
metrics={'train_runtime': 2874.2777, 'train_samples_per_second':
```

```
val_results = trainer.evaluate(val_tok)
test_results = trainer.evaluate(test_tok)

print("Validation Results:", val_results)
print("Test Results:", test_results)
```

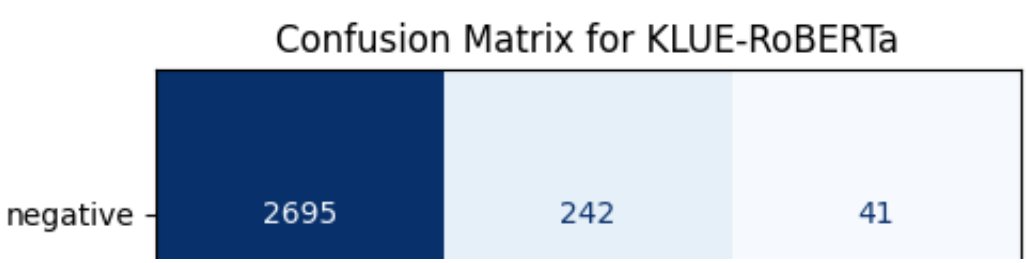
[313/313 01:14]

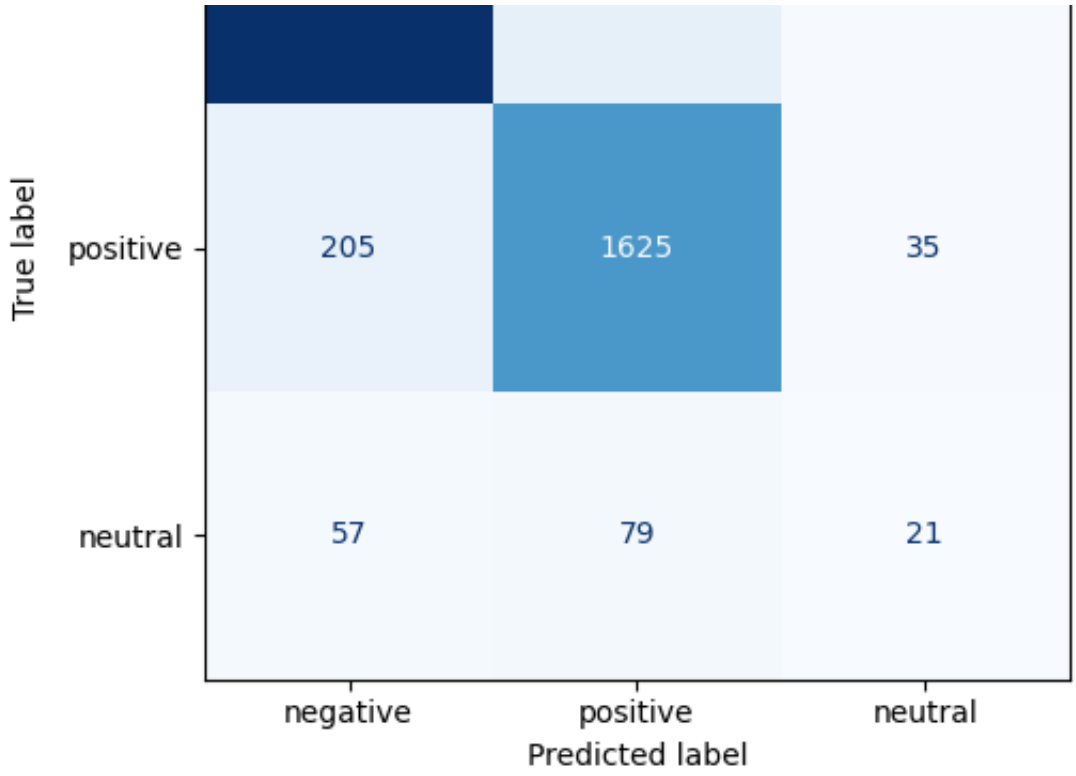
```
Validation Results: {'eval_loss': 0.5472621321678162, 'eval_accuracy':
Test Results: {'eval_loss': 0.5196225643157959, 'eval_accuracy': 0.8682
```

```
klue_test_preds_output = trainer.predict(test_tok)
klue_test_preds = np.argmax(klue_test_preds_output.predictions, axis=1
klue_test_labels = klue_test_preds_output.label_ids

print(classification_report(klue_test_labels, klue_test_preds, target_
print(confusion_matrix(klue_test_labels, klue_test_preds))
```

```
              precision    recall  f1-score   support

    negative       0.91      0.90      0.91      2978
    positive       0.84      0.87      0.85      1865
     neutral       0.22      0.13      0.17       157

    accuracy                           0.87      5000
   macro avg       0.65      0.64      0.64      5000
weighted avg       0.86      0.87      0.86      5000

[[2695  242   41]
 [ 205 1625   35]
 [  57   79   21]]
```

```
from sklearn.metrics import ConfusionMatrixDisplay

cm = np.array([
    [2695, 242, 41],
    [205, 1625, 35],
    [57, 79, 21]
])

label_names_3 = ["negative", "positive", "neutral"]

fig, ax = plt.subplots(figsize=(6, 5))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=label
disp.plot(ax=ax, values_format="d", cmap="Blues", colorbar=False)

plt.title("Confusion Matrix for KLUE-RoBERTa")
plt.tight_layout()
plt.show()
```






```
np.save("klue_test_preds.npy", klue_test_preds)
np.save("klue_test_labels.npy", klue_test_labels)
```

```
np.save("/content/drive/MyDrive/klue_test_preds.npy", klue_test_preds
np.save("/content/drive/MyDrive/klue_test_labels.npy", klue_test_labe
```

```
Start coding or generate with AI.
```

```
['.config', 'xlmr_results', 'mbert_results', 'df_test_processed.csv',
['Colab Notebooks', 'Getting started.pdf', 'SchoolWork', 'Screen Shot
```

8. Multilingual BERT (mBERT)

This section fine-tunes multilingual BERT on the same task to compare multilingual pretraining with Korean-specific pretraining.

Start coding or generate with AI.

```
model_name = "bert-base-multilingual-cased"
tokenizer = AutoTokenizer.from_pretrained(model_name)

train_tok = train_hf.map(tokenize_function, batched=True)
val_tok = val_hf.map(tokenize_function, batched=True)
test_tok = test_hf.map(tokenize_function, batched=True)

train_tok = train_tok.rename_column("y", "labels")
val_tok = val_tok.rename_column("y", "labels")
test_tok = test_tok.rename_column("y", "labels")

train_tok.set_format(type="torch", columns=["input_ids", "attention_ma
val_tok.set_format(type="torch", columns=["input_ids", "attention_masl
test_tok.set_format(type="torch", columns=["input_ids", "attention_mas
```

```
Map: 100%  40000/40000 [00:11<00:00, 6962.67 examples/s]

Map: 100%  5000/5000 [00:00<00:00, 7093.10 examples/s]

Map: 100%  5000/5000 [00:00<00:00, 4865.83 examples/s]
```

```
model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=3
)

training_args = TrainingArguments(
    output_dir="./mbert_results",
    eval_strategy="epoch",
    save_strategy="epoch",
    logging_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=3,
    weight_decay=0.01,
    load_best_model_at_end=True,
    metric_for_best_model="macro_f1",
    greater_is_better=True,
    report_to="none"
)
```

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_tok,
    eval_dataset=val_tok,
    compute_metrics=compute_metrics
)

trainer.train()
```

model.safetensors: 100% 714M/714M [00:08<00:00, 82.1MB/s]

Loading weights: 100% 199/199 [00:00<00:00, 612.42it/s, Materializing param=bert.pooler.dense.weig

```
BertForSequenceClassification LOAD REPORT from: bert-base-multilingual-
Key                                          | Status     |
---------------------------------------------+------------+-
cls.seq_relationship.weight                  | UNEXPECTED |
cls.predictions.bias                         | UNEXPECTED |
cls.predictions.transform.dense.weight       | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight   | UNEXPECTED |
cls.predictions.transform.dense.bias         | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias     | UNEXPECTED |
cls.seq_relationship.bias                    | UNEXPECTED |
classifier.weight                            | MISSING    |
classifier.bias                              | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/archit
- MISSING       :those params were newly initialized because missing fr
```

[7281/7500 45:09 < 01:21, 2.69 it/s, Epoch 2.91/3]

Epoch	Training Loss	Validation Loss	Accuracy	Macro F1
1	0.564002	0.532734	0.802600	0.533717
2	0.442814	0.518229	0.805200	0.539492

Writing model shards: 100% 1/1 [00:03<00:00, 3.07s/it]

```
Writing model shards:   0%|          | 0/1 [00:00<?, ?it/s]
```

[7500/7500 47:18, Epoch 3/3]

Epoch	Training Loss	Validation Loss	Accuracy	Macro F1
1	0.564002	0.532734	0.802600	0.533717
2	0.442814	0.518229	0.805200	0.539492

3	0.347303	0.544853	0.810200	0.546527

Writing model shards: 100% 1/1 [00:08<00:00, 8.36s/it]

```
There were missing keys in the checkpoint model loaded: ['bert.embeddir
There were unexpected keys in the checkpoint model loaded: ['bert.embec
TrainOutput(global_step=7500, training_loss=0.45137298990885416,
```

```
val_results = trainer.evaluate(val_tok)
test_results = trainer.evaluate(test_tok)

print("Validation Results:", val_results)
print("Test Results:", test_results)
```

[313/313 01:11]

```
Validation Results: {'eval_loss': 0.5448525547981262, 'eval_accuracy':
Test Results: {'eval_loss': 0.5325157046318054, 'eval_accuracy': 0.821,
```

```
mbert_test_preds_output = trainer.predict(test_tok)
mbert_test_preds = np.argmax(mbert_test_preds_output.predictions, axis
mbert_test_labels = mbert_test_preds_output.label_ids

label_names_3 = ["negative", "positive", "neutral"]

print(classification_report(mbert_test_labels, mbert_test_preds, targe
print(confusion_matrix(mbert_test_labels, mbert_test_preds))
```

```
              precision    recall  f1-score   support

    negative       0.86      0.87      0.87      2978
    positive       0.77      0.81      0.79      1865
     neutral       0.00      0.00      0.00       157

    accuracy                           0.82      5000
   macro avg       0.54      0.56      0.55      5000
weighted avg       0.80      0.82      0.81      5000

[[2600  377    1]
 [ 358 1505    2]
 [  73   84    0]]
```

9. XLM-RoBERTa

This section fine-tunes XLM-RoBERTa as a stronger multilingual baseline.

```
model_name = "xlm-roberta-base"
tokenizer = AutoTokenizer.from_pretrained(model_name)

train_tok = train_hf.map(tokenize_function, batched=True)
val_tok = val_hf.map(tokenize_function, batched=True)
test_tok = test_hf.map(tokenize_function, batched=True)

train_tok = train_tok.rename_column("y", "labels")
val_tok = val_tok.rename_column("y", "labels")
test_tok = test_tok.rename_column("y", "labels")

train_tok.set_format(type="torch", columns=["input_ids", "attention_ma
val_tok.set_format(type="torch", columns=["input_ids", "attention_masl
test_tok.set_format(type="torch", columns=["input_ids", "attention_ma
```

```
config.json: 100%                                   615/615 [00:00<00:00, 17.9kB/s]

tokenizer_config.json: 100%                         25.0/25.0 [00:00<00:00, 1.95kB/s]

sentencepiece.bpe.model: 100%                       5.07M/5.07M [00:00<00:00, 17.0MB/s]

tokenizer.json:        9.10M/? [00:00<00:00, 19.8MB/s]

Map: 100%                                           40000/40000 [00:23<00:00, 2614.94 examples/s]

Map: 100%                                           5000/5000 [00:02<00:00, 1667.76 examples/s]

Map: 100%                                           5000/5000 [00:01<00:00, 3410.18 examples/s]
```

```
model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=3
)

training_args = TrainingArguments(
    output_dir="./xlmr_results",
    eval_strategy="epoch",
    save_strategy="epoch",
```

```
    logging_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=3,
    weight_decay=0.01,
    load_best_model_at_end=True,
    metric_for_best_model="macro_f1",
    greater_is_better=True,
    report_to="none"
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_tok,
    eval_dataset=val_tok,
    compute_metrics=compute_metrics
)

trainer.train()
```

model.safetensors: 100% 1.12G/1.12G [00:14<00:00, 84.0MB/s]

Loading weights: 100% 197/197 [00:01<00:00, 199.20it/s, Materializing param=roberta.encoder.layer.

```
XLMRobertaForSequenceClassification LOAD REPORT from: xlm-roberta-base
Key                           | Status     |
-----------------------------+------------+-
lm_head.layer_norm.bias       | UNEXPECTED |
lm_head.bias                  | UNEXPECTED |
roberta.pooler.dense.bias     | UNEXPECTED |
roberta.pooler.dense.weight   | UNEXPECTED |
lm_head.dense.bias            | UNEXPECTED |
lm_head.dense.weight          | UNEXPECTED |
lm_head.layer_norm.weight     | UNEXPECTED |
classifier.out_proj.bias      | MISSING    |
classifier.out_proj.weight    | MISSING    |
classifier.dense.bias         | MISSING    |
classifier.dense.weight       | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/archit
- MISSING       :those params were newly initialized because missing fr
```

[7500/7500 51:59, Epoch 3/3]

Epoch	Training Loss	Validation Loss	Accuracy	Macro F1

1	0.510774	0.483751	0.841200	0.560670
2	0.390169	0.414599	0.852600	0.572236
3	0.315383	0.458165	0.853000	0.572932

Writing model shards: 100% 1/1 [00:21<00:00, 21.18s/it]

Writing model shards: 100% 1/1 [00:21<00:00, 21.83s/it]

Writing model shards: 100% 1/1 [00:05<00:00, 5.64s/it]

```
There were missing keys in the checkpoint model loaded: ['roberta.embec
There were unexpected keys in the checkpoint model loaded: ['roberta.em
TrainOutput(global_step=7500, training_loss=0.4054421223958333,
metrics={'train_runtime': 3120.0574, 'train_samples_per_second':
```

```
val_results = trainer.evaluate(val_tok)
test_results = trainer.evaluate(test_tok)

print("Validation Results:", val_results)
print("Test Results:", test_results)
```

[313/313 01:07]

```
Validation Results: {'eval_loss': 0.4581652283668518, 'eval_accuracy':
Test Results: {'eval_loss': 0.4670892655849457, 'eval_accuracy': 0.8498
```

```
xlmr_test_preds_output = trainer.predict(test_tok)
xlmr_test_preds = np.argmax(xlmr_test_preds_output.predictions, axis=1
xlmr_test_labels = xlmr_test_preds_output.label_ids

print(classification_report(xlmr_test_labels, xlmr_test_preds, target_
print(confusion_matrix(xlmr_test_labels, xlmr_test_preds))
```

```
              precision    recall  f1-score   support

    negative       0.89      0.89      0.89      2978
    positive       0.79      0.86      0.82      1865
     neutral       0.00      0.00      0.00       157

    accuracy                           0.85      5000
   macro avg       0.56      0.58      0.57      5000
weighted avg       0.83      0.85      0.84      5000

[[2643  335    0]
 [ 259 1606    0]
 [  68   89    0]]
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classificatior
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result)
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classificatior
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result)
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classificatior
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result)
```

```
np.save("/content/drive/MyDrive/mbert_test_preds.npy", mbert_test_pre
np.save("/content/drive/MyDrive/mbert_test_labels.npy", mbert_test_lal
np.save("/content/drive/MyDrive/xlmr_test_preds.npy", xlmr_test_preds
np.save("/content/drive/MyDrive/xlmr_test_labels.npy", xlmr_test_labe

df_train.to_csv("/content/drive/MyDrive/df_train_processed.csv", inde
df_val.to_csv("/content/drive/MyDrive/df_val_processed.csv", index=Fa
df_test.to_csv("/content/drive/MyDrive/df_test_processed.csv", index=
```

```
klue_test_preds = np.load('/content/drive/MyDrive/klue_test_preds.npy
klue_test_labels = np.load('/content/drive/MyDrive/klue_test_labels.n

mbert_test_preds = np.load('/content/drive/MyDrive/mbert_test_preds.n
mbert_test_labels = np.load('/content/drive/MyDrive/mbert_test_labels

xlmr_test_preds = np.load('/content/drive/MyDrive/xlmr_test_preds.npy
xlmr_test_labels = np.load('/content/drive/MyDrive/xlmr_test_labels.n

print(klue_test_preds[:10])
print(klue_test_labels[:10])
```

```
[1 0 1 0 0 0 1 1 0 0]
[1 0 1 0 0 0 1 1 0 0]
```

10. Neutral-Class Error Analysis (Using KLUE-RoBERTa)

This section examines why the neutral class remains difficult even for the strongest model, KLUE-RoBERTa.

```
df_test_analysis = df_test.copy().reset_index(drop=True)
df_test_analysis["pred"] = klue_test_preds
df_test_analysis["true_label"] = klue_test_labels

id_to_sentiment = {0: "negative", 1: "positive", 2: "neutral"}

df_test_analysis["true_sentiment"] = df_test_analysis["true_label"].ma
df_test_analysis["pred_sentiment"] = df_test_analysis["pred"].map(id_
```

```
neutral_true = df_test_analysis[df_test_analysis["true_sentiment"] ==

print("Number of true neutral examples:", len(neutral_true))
print(neutral_true["pred_sentiment"].value_counts())
```

```
Number of true neutral examples: 157
pred_sentiment
positive    79
negative    57
neutral     21
Name: count, dtype: int64
```

```
def label_ids_to_names(label_ids):
    return [id_to_label[i] for i in label_ids]

neutral_true["label_names"] = neutral_true["labels"].apply(label_ids_
```

```
neutral_as_negative = neutral_true[neutral_true["pred_sentiment"] == '
neutral_as_positive = neutral_true[neutral_true["pred_sentiment"] == '
```

```
for i, row in neutral_as_positive.head(10).iterrows():
    print("TEXT:", row["text"])
    print("LABEL NAMES:", row["label_names"])
    print()
```

```
TEXT: 추장관님 적극지지합니다. . 처갓집  검사가 대한민국을 더 이상 흔들지 못하게 해주세요!
LABEL NAMES: ['환영/호의', '지긋지긋', '화남/분노', '존경', '기대감', '비장함', '의

TEXT: 저것의 이름은 카라멜 수제 가죽 케이스라고 합니다.
LABEL NAMES: ['우쭐댐/무시함', '없음', '깨달음', '놀람']

TEXT: 존나 괄호치니까 하이라이트효과 ㅋㅋㅋㅋㅋㅋ
LABEL NAMES: ['감동/감탄', '어이없음', '즐거움/신남', '깨달음', '당황/난처']

TEXT: 오늘 같은 날은 푹 쉬어야 되어요!!
LABEL NAMES: ['환영/호의', '기대감', '비장함', '신기함/관심', '아껴주는', '짜증',

TEXT: 요즘도마트에서 팔까요찰떡아이스 나가본지가하도오래되서
LABEL NAMES: ['기대감', '의심/불신', '신기함/관심', '불안/걱정']

TEXT: 알못인데 스타인웨이? 가 콩쿨에서 우승 많이하는 피아노란것만 앎..
LABEL NAMES: ['신기함/관심', '없음', '깨달음', '재미없음']

TEXT: 좋은 현상임부디 이준석을 기점으로 기존 정치, 국회의원들 사고 방식이 달라졌으면 좋겠음!
LABEL NAMES: ['불평/불만', '감동/감탄', '기대감', '안타까움/실망', '신기함/관심', '

TEXT: 생각보다 괜찮앗어요 ㅎㅎ 부드러운걸 생각햇는데ㅠㅠ 딱딱하네욜
LABEL NAMES: ['불평/불만', '기대감', '안타까움/실망', '편안/쾌적', '당황/난처', '안

TEXT: 헐 뭐야? 고맙네??????? (어리둥절
LABEL NAMES: ['감동/감탄', '고마움', '신기함/관심', '어이없음', '깨달음', '당황/난처

TEXT: 아직 다 먹어보진않았는데 괜찮은것 같아요 구성을 고를 수가 없어서 아쉬워요
LABEL NAMES: ['기대감', '안타까움/실망']
```

```
counter_neg = Counter()
for labels in neutral_as_negative["label_names"]:
    counter_neg.update(labels)

counter_pos = Counter()
for labels in neutral_as_positive["label_names"]:
    counter_pos.update(labels)

print("Most common label names in neutral->negative errors:")
print(counter_neg.most_common(20))

print("\nMost common label names in neutral->positive errors:")
print(counter_pos.most_common(20))
```

```
Most common label names in neutral->negative errors:
[('신기함/관심', 26), ('안타까움/실망', 22), ('깨달음', 22), ('기대감', 22), ('우

Most common label names in neutral->positive errors:
[('기대감', 32), ('없음', 31), ('신기함/관심', 30), ('깨달음', 29), ('당황/난처'
```

```
neutral_source_labels = {'우쭐댐/무시함', '비장함', '없음', '깨달음', '놀람'}
```

```
counter_neutral_source_neg = Counter()
for labels in neutral_as_negative["label_names"]:
    for lab in labels:
        if lab in neutral_source_labels:
            counter_neutral_source_neg[lab] += 1

counter_neutral_source_pos = Counter()
for labels in neutral_as_positive["label_names"]:
    for lab in labels:
        if lab in neutral_source_labels:
            counter_neutral_source_pos[lab] += 1

print("Neutral-source labels in neutral->negative errors:")
print(counter_neutral_source_neg)

print("\nNeutral-source labels in neutral->positive errors:")
print(counter_neutral_source_pos)
```

```
Neutral-source labels in neutral->negative errors:
Counter({'깨달음': 22, '우쭐댐/무시함': 19, '없음': 12, '놀람': 10, '비장함': 1

Neutral-source labels in neutral->positive errors:
Counter({'없음': 31, '깨달음': 29, '놀람': 21, '우쭐댐/무시함': 15, '비장함': 1
```

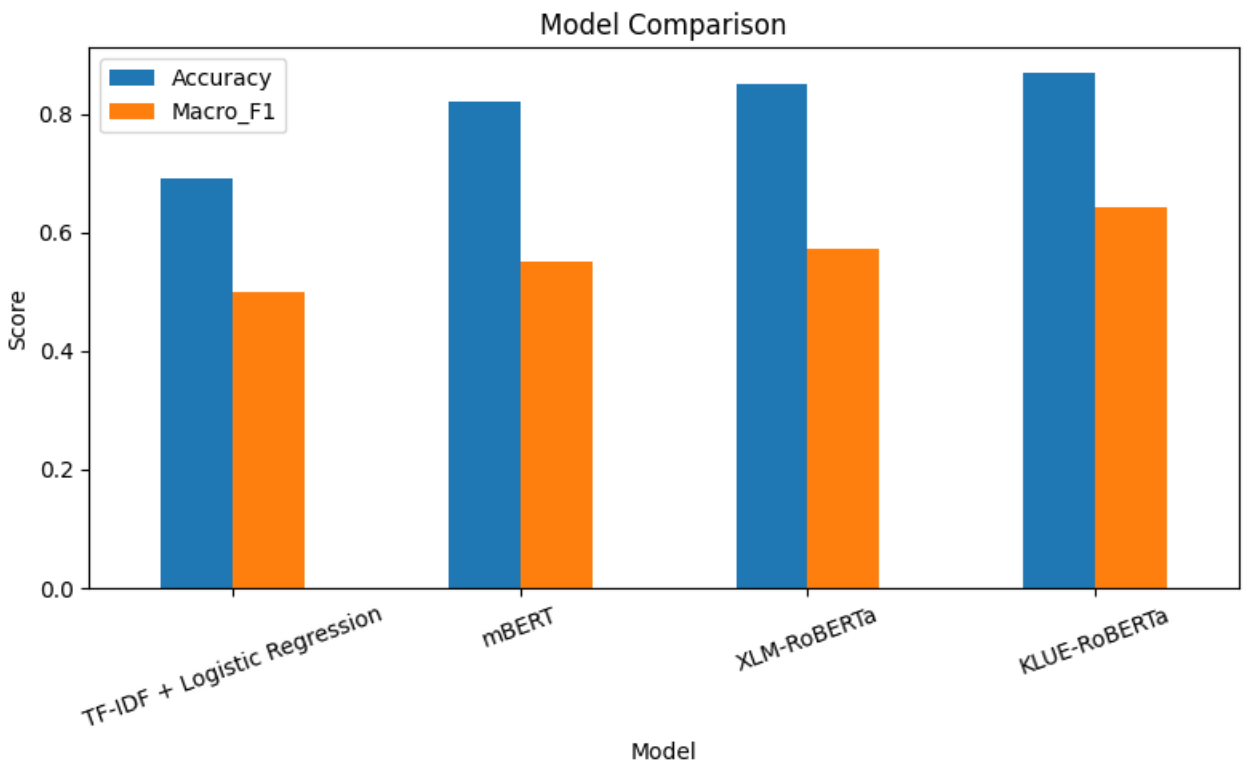
11. Final Model Comparison Table

This section summarizes the performance of all models.

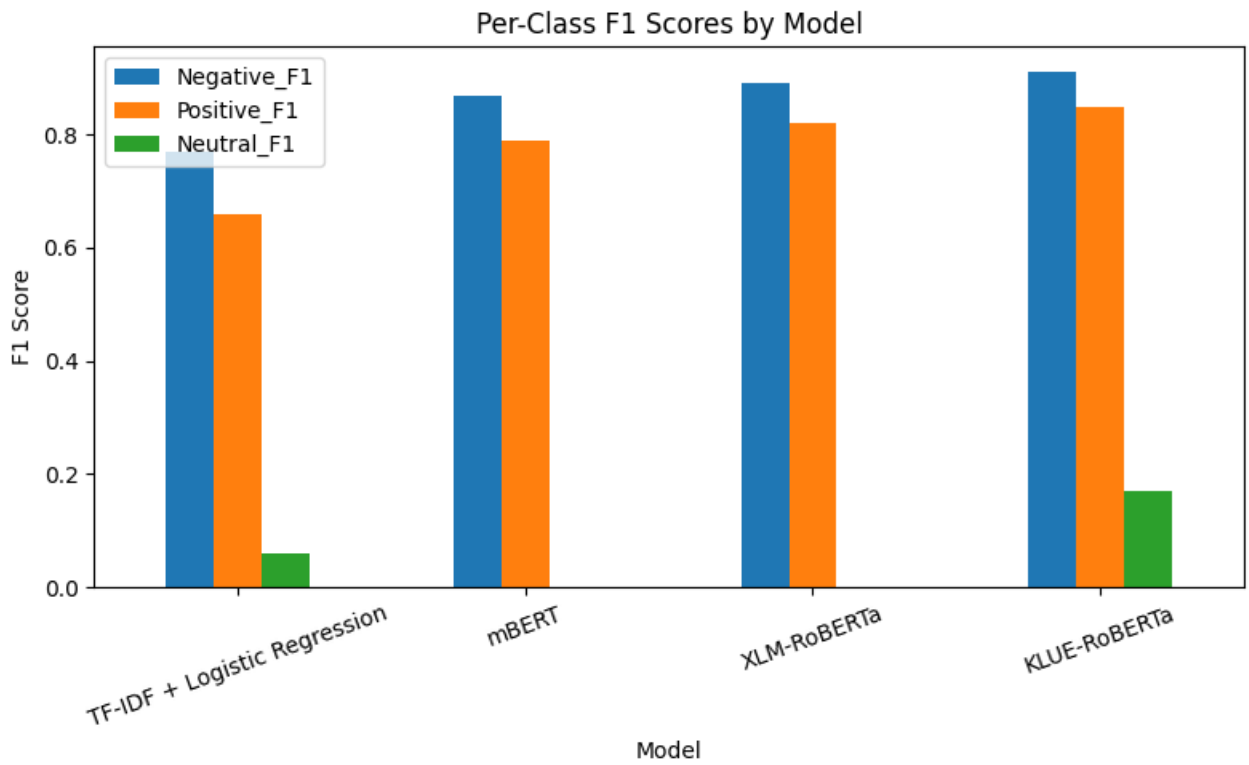
```
results_summary = pd.DataFrame({
    "Model": [
        "TF-IDF + Logistic Regression",
        "mBERT",
        "XLM-RoBERTa",
        "KLUE-RoBERTa"
    ],
    "Accuracy": [0.690, 0.821, 0.850, 0.868],
    "Macro_F1": [0.499, 0.550, 0.571, 0.642],
    "Negative_F1": [0.77, 0.87, 0.89, 0.91],
    "Positive_F1": [0.66, 0.79, 0.82, 0.85],
    "Neutral_F1": [0.06, 0.00, 0.00, 0.17]
})

print(results_summary)
```

```
                         Model  Accuracy  Macro_F1  Negative_F1  Posi
0  TF-IDF + Logistic Regression     0.690     0.499         0.77
1                        mBERT     0.821     0.550         0.87
2                  XLM-RoBERTa     0.850     0.571         0.89
3                 KLUE-RoBERTa     0.868     0.642         0.91

   Neutral_F1
0        0.06
1        0.00
2        0.00
3        0.17
```

```
results_summary.plot(x="Model", y=["Accuracy", "Macro_F1"], kind="bar'
plt.title("Model Comparison")
plt.ylabel("Score")
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()
```



```
results_summary.plot(
    x="Model",
    y=["Negative_F1", "Positive_F1", "Neutral_F1"],
    kind="bar",
    figsize=(8, 5)
)

plt.title("Per-Class F1 Scores by Model")
plt.ylabel("F1 Score")
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()
```



Appendix. Translation Model

```
trans_model_name = "Helsinki-NLP/opus-mt-ko-en"

trans_tokenizer = AutoTokenizer.from_pretrained(trans_model_name)
trans_model = AutoModelForSeq2SeqLM.from_pretrained(trans_model_name)
```

```
config.json:        1.39k/? [00:00<00:00, 29.3kB/s]

tokenizer_config.json: 100%                                   44.0/44.0 [00:00<00:00, 1.69kB/s]

source.spm: 100%                                              842k/842k [00:00<00:00, 16.0MB/s]

target.spm: 100%                                              813k/813k [00:00<00:00, 15.3MB/s]

vocab.json:         1.72M/? [00:00<00:00, 49.0MB/s]

pytorch_model.bin: 100%                                       312M/312M [00:06<00:00, 44.7MB/s]

Loading weights: 100%   258/258 [00:00<00:00, 634.39it/s, Materializing param=model.shared.weight]

model.safetensors: 100%                                       312M/312M [00:03<00:00, 178MB/s]
The tied weights mapping and config for this model specifies to tie moc
The tied weights mapping and config for this model specifies to tie moc
generation_config.json: 100%                                  293/293 [00:00<00:00, 33.1kB/s]
```

```
import torch

device = "cuda" if torch.cuda.is_available() else "cpu"
trans_model = trans_model.to(device)
print(device)
```

```
cuda
```

```
def translate_texts(texts, batch_size=16, max_input_length=128, max_out
    translated_texts = []

    for i in range(0, len(texts), batch_size):
        batch = texts[i:i+batch_size]

        inputs = trans_tokenizer(
            batch,
            return_tensors="pt",
            padding=True,
            truncation=True,
            max_length=max_input_length
        ).to(device)

        outputs = trans_model.generate(
            **inputs,
            max_length=max_output_length
        )

        decoded = trans_tokenizer.batch_decode(outputs, skip_special_to
        translated_texts.extend(decoded)

        if i % 500 == 0:
            print(f"Translated {i} / {len(texts)}")

    return translated_texts
```

```
sample_translations = translate_texts(df_train["text"].tolist()[:10], b
for i in range(5):
    print("KO:", df_train["text"].iloc[i])
    print("EN:", sample_translations[i])
    print()
```

```
Translated 0 / 10
KO: 내가 톰행크스를 좋아하긴 했나보다... 초기 영화 빼고는 다 봤네.
EN: I don't know if I liked Tom Hanks... except for the early movies.

KO: 정말 상상을 초월하는 무개념 진상들 상대하다 우울증, 공항장애 걸리는 공무원 많아요 ㅠ 생
EN: I think there's a lot of people in the public sector who are dealir

KO: 새로운 세상과 조우한 자의 어린아이 같은 반응, 어쩌면 회복된 것은 눈이 아닌 순수함일지도
EN: The new world and the child-like response of the friendly, perhaps

KO: 미역은 원생생물계 산호초는 동물ㅇㅇ 아 미역이 바다의 새ㄱㅇㄱㅋㅋㅋㅋㅋㅋㅋㅋㅋㅋㅋ
EN: The cyto-living coral reef is an animal, and the micro world is a b

KO: 네 맞습니다 플스는 역시 30프레임이 어울리죠 ㅎ
EN: That's right. Flies also fit in 30 frames.
```